The Boss' Challenge

A few days ago, your boss learned about the attr_accessor method that you read about in *The attr_accessor() Example*, on page 116. Now she's using attr_accessor all the time to generate her objects' attributes. While she was at it, your boss also came up with the idea of writing her own *Class Macro (117)*, similar to attr_accessor, which generates a *validated* attribute. "I call it attr_checked," she says.

Your boss explains how this attr_checked method should work, pointing out that it should take the name of the attribute, as well as a block. The block is used for validation. If you assign a value to the attribute and the block doesn't return true for that value, then you get a runtime exception.

Your boss' first requirement is an attr_checked Class Macro, and she explains her secondary requirement: "I don't want this attr_checked method to be available to each and every class, because I don't like the idea of cluttering standard classes with my own methods. Instead, a class should gain access to attr_checked only when it includes a CheckedAttributes module." She provides this example:

```
class Person
  include CheckedAttributes

  attr_checked :age do |v|
    v >= 18
  end
end

me = Person.new
me.age = 39     # OK
me.age = 12     # Exception
```

Your task today is to write CheckedAttributes and attr_checked for your boss.

A Development Plan

The boss' challenge is a bit too much to handle in a single burst of coding. You'll get to a solution in small steps.

Instead of engaging in pair programming, Bill proposes sharing roles: he'll manage the development, and you'll write the code. While you wonder what "managing the development" actually means, Bill quickly lists the steps you'll take:

1. Write a *Kernel Method (32)* named add_checked_attribute using eval to add a super-simple validated attribute to a class.

2. Refactor add_checked_attribute to remove eval.